

Modelos de Deep Learning

Redes Neuronales Recurrentes

Universidad ORT Uruguay

3 de Noviembre, 2025

Redes Neuronales Recurrentes (RNNs)

- Las **RNNs** están diseñadas para procesar datos **secuenciales**
 - texto
 - voz
 - series temporalesdonde el **orden de los elementos** es importante.
- A diferencia de las feedforward las RNNs utilizan **conexiones recurrentes**.
- La salida en un instante de tiempo se retroalimenta como entrada a la red en el siguiente instante.
- Esto permite que **capturen dependencias temporales**.

Unidad recurrente y estado oculto

- El componente fundamental de una RNN es la **unidad recurrente**
- Esta mantiene un **estado oculto** -una forma de memoria- que se actualiza en cada paso temporal.
- Esta actualización depende
 - **entrada actual**
 - **estado oculto anterior**
- Esta retroalimentación permite que la red **aprenda de entradas pasadas** y **utilice ese conocimiento** en el procesamiento presente.

Definición formal de una RNN

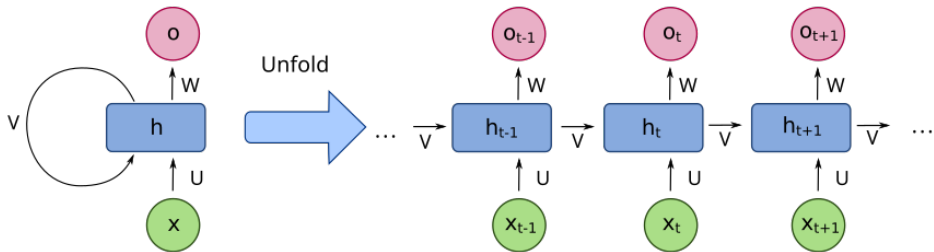
- Una RNN puede representarse como una función

$$f_{\theta} : (\mathbf{x}_t, \mathbf{h}_t) \mapsto (\mathbf{y}_t, \mathbf{h}_{t+1}),$$

donde:

- $\mathbf{x}_t$: vector de entrada;
 - $\mathbf{h}_t$: vector oculto;
 - $\mathbf{y}_t$: vector de salida;
 - θ : parámetros de la red neuronal.
-
- Transforma una **entrada** $\mathbf{x}_t$ en una **salida** $\mathbf{y}_t$, utilizando el **vector oculto** $\mathbf{h}_t$ como una forma de **memoria**.
 - En cada paso **actualiza su memoria**.

RNN unfolding



Ecuaciones de una RNN

- El estado oculto y la salida en el instante t se actualizan según:

$$\mathbf{h}_t = \text{Activation}(\mathbf{U}\mathbf{x}_t + \mathbf{V}\mathbf{h}_{t-1} + \mathbf{b}_h)$$

$$\mathbf{y}_t = \text{Activation}(\mathbf{W}\mathbf{h}_t + \mathbf{b}_y)$$

donde:

- $\mathbf{U}$: matriz de pesos entre la entrada y el estado oculto;
 - $\mathbf{V}$: matriz de pesos recurrente (de $\mathbf{h}_{t-1}$ a $\mathbf{h}_t$);
 - $\mathbf{W}$: matriz de pesos de salida;
 - $\mathbf{b}_h, \mathbf{b}_y$: vectores de sesgo;
- En forma abstracta:

$$f_{\theta} : (\mathbf{x}_t, \mathbf{h}_{t-1}) \mapsto (\mathbf{y}_t, \mathbf{h}_t), \quad \theta = (\mathbf{W}, \mathbf{U}, \mathbf{V}, \mathbf{b}_h, \mathbf{b}_y).$$

Ventajas y desventajas de las RNNs

Ventajas

- Posibilidad de procesar entradas de cualquier longitud.
- El tamaño del modelo no aumenta con el tamaño de la entrada.
- El cómputo tiene en cuenta la información histórica.
- Los pesos se comparten a lo largo del tiempo.

Desventajas

- El cómputo puede ser lento.
- Dificultad para acceder a información de pasos muy anteriores.
- No puede considerar entradas futuras para el estado actual.

Función de pérdida en una RNN

- Cada paso t produce una predicción $\hat{\mathbf{y}}_t$ y se calcula una pérdida **local**:

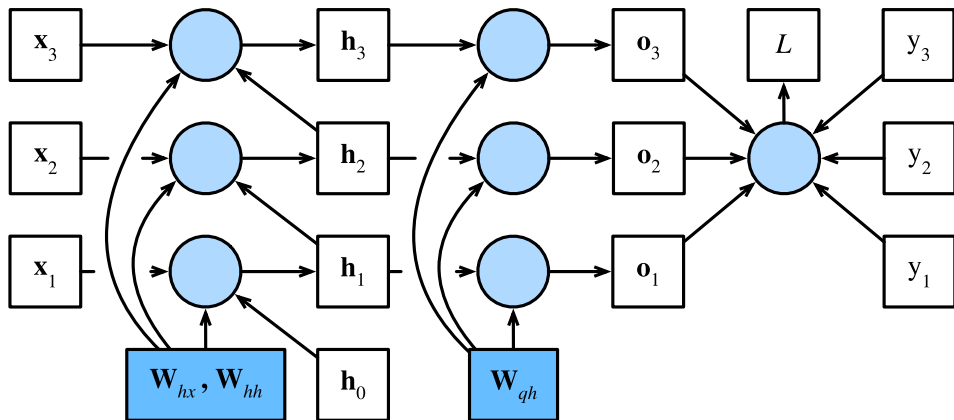
$$\text{Loss}(\hat{\mathbf{y}}_t, \mathbf{y}_t).$$

- La **pérdida total** de la secuencia combina los errores de todos los pasos:

$$\text{Loss}(\hat{\mathbf{y}}, \mathbf{y}) = \sum_{t=1}^{T_y} \text{Loss}(\hat{\mathbf{y}}_t, \mathbf{y}_t)$$

- Cada término mide la diferencia de la salida con la real en el instante t .
- Esta suma refleja que la red debe aprender a producir salidas coherentes a lo largo de **toda la secuencia**, no solo en el último paso.

Retropropagación a través del tiempo (BPTT)



Idea general de BPTT

- La **BPTT** consiste en:
 - **Desplegar** la red f a lo largo de varios pasos temporales, creando copias consecutivas de f que comparten los mismos pesos.
 - Aplicar la **BP clásico** (como en una feedforward) sobre el grafo desplegado.
- En cada paso t , la red procesa:

$$(\hat{\mathbf{y}}_t, \mathbf{h}_t) = f(\mathbf{x}_t, \mathbf{h}_{t-1}),$$

y la pérdida total acumula los errores de todos los instantes.

- El gradiente final resulta de sumar las contribuciones de todas las copias

$$\frac{\partial \text{Loss}^{(T)}}{\partial W} = \sum_{t=1}^T \left. \frac{\partial \text{Loss}^{(T)}}{\partial W} \right|_{(t)}$$

Horizonte temporal (k)

- k define el **horizonte temporal** sobre el cual se despliega la red.
- En cada iteración se considera un segmento de longitud k de la secuencia:

$$(\mathbf{x}_t, \mathbf{x}_{t+1}, \dots, \mathbf{x}_{t+k-1}),$$

sobre el cual se aplica la retropropagación.

- Dos casos relevantes:
 - **BPTT completa**: $k = n$ (toda la secuencia). Se obtiene un gradiente exacto pero costoso e inestable para secuencias largas.
 - **BPTT truncada**: $k < n$. Se retropropaga sólo k pasos atrás, reduciendo el costo y evitando **desvanecimiento o explosión de gradientes**.
- El valor de k controla la **profundidad temporal del aprendizaje**.

Algorithm Back Propagation Through Time

- 1: $\mathbf{x}_t$ entrada en tiempo t , $\mathbf{y}_t$ la salida esperada
 - 2: Desplegar la red para contener k copias de la función f
 - 3: **repeat**
 - 4: $\mathbf{h}_0 \leftarrow \mathbf{0}$ (estado inicial)
 - 5: **for** $t = 0$ **hasta** $n - k$ **do** ($n =$ longitud de la secuencia)
 - 6: Establecer las entradas de la red en $(\mathbf{h}_t, \mathbf{x}_t, \mathbf{x}_{t+1}, \dots, \mathbf{x}_{t+k-1})$
 - 7: Propagar hacia adelante en toda la red desplegada:
$$(\hat{\mathbf{y}}_{t+i}, \mathbf{h}_{t+i}) = f(\mathbf{x}_{t+i}, \mathbf{h}_{t+i-1})$$
 - 8: Retropropagar el error a través de las k copias de la red
 - 9: Acumular los gradientes $\Delta W, \Delta U, \Delta V$ de las k instancias
 - 10: Actualizar los pesos: $(W, U, V) \leftarrow (W, U, V) + \eta(\Delta W, \Delta U, \Delta V)$
 - 11: **end for**
 - 12: **until** se cumple el criterio de parada
-

RNNs: Problemas

■ Gradientes

- **Exploding gradients:** gradientes que crecen exponencialmente.
- **Vanishing gradients:** gradientes que disminuyen a valores extremadamente cercanos a 0.

■ Entrenamiento

- RNNs *vanilla* (tradicionales, sin ninguna modificación) tienden a no lograr aprender dependencias temporales muy largas.

Un Ejemplo Concreto: Copying Task

- Benchmark que demuestra empíricamente la **limitación de las RNNs vanilla** para retener memoria a largo plazo.
- Diseñada para evaluar la **capacidad de memoria pura**.
- La red debe almacenar una secuencia de S símbolos y, tras un retardo de T pasos, **reproducirla exactamente**.
- Se mide la pérdida promedio (entropía cruzada) en cada paso temporal.

Definición del experimento

- Longitud total de la secuencia de entrada: $T + 2S$
- Alfabeto: $A = \{a_1, \dots, a_K, a_{K+1}, a_{K+2}\}$ donde:
 - a_{K+1} = símbolo en blanco (blank)
 - a_{K+2} = símbolo delimitador (delimiter)
- Estructura de la entrada:
 1. Secuencia a recordar (S pasos): símbolos de $\{a_i\}_{i=1}^K$.
 2. Brecha de retardo ($T - 1$ pasos): sólo el símbolo en blanco.
 3. Delimitador (1 paso): indica el inicio de la reproducción.
 4. Entradas finales (S pasos): símbolos en blanco.
- La red debe mantener su salida en el símbolo blank durante los primeros $T + S$ pasos. Después de recibir el delimitador, debe reproducir exactamente los S símbolos originales en el mismo orden en que aparecieron al inicio de la secuencia.

Parámetros críticos y umbral de fallo

■ Variables experimentales:

Parámetro	Símbolo	Valores típicos
Alfabeto principal	K	8–10 símbolos
Longitud de secuencia	S	10 pasos
Longitud de brecha	T	500 o 1000 pasos

- Si la red falla, su pérdida converge al valor de adivinación aleatoria:

$$L_{\text{rand}} = \ln(K + 2).$$

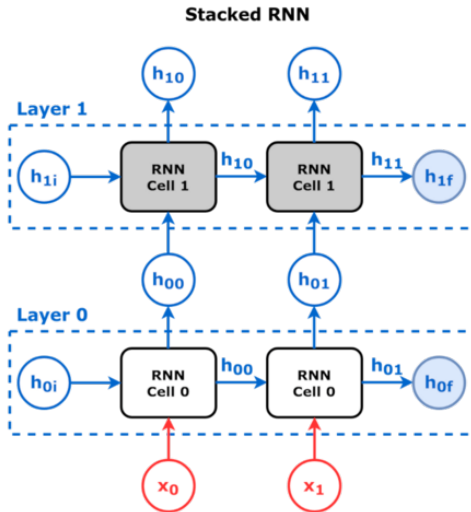
- Para $K = 8$, $L_{\text{rand}} = \ln(10) \approx 2.30$.
- Si $\text{Loss} \approx L_{\text{rand}}$, $\mathbf{h}_t$ ha perdido toda información de la secuencia original.

Resultados típicos en la Tarea de Copia ($T = 1000$)

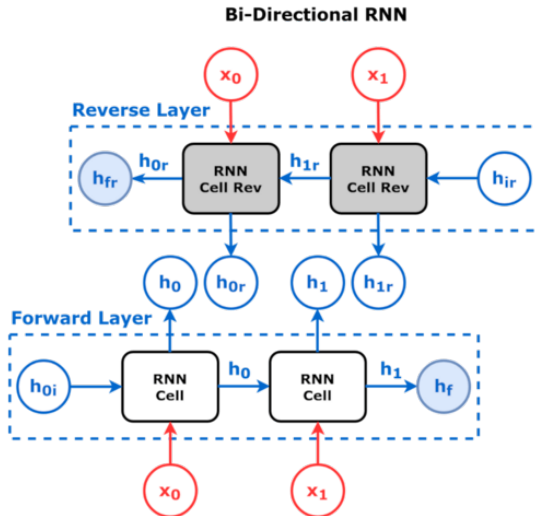
Arquitectura	Pérdida de Entropía Cruzada (Típica)	Conclusión de Capacidad de Memoria
RNN Vanilla Estándar	$\approx \ln(K + 2)$ (Alto Error)	Falla; converge al rendimiento de adivinación aleatoria.
LSTM	≈ 0.0 (Bajo Error)	Éxito; conserva la memoria perfectamente.
GRU	≈ 0.0 (Bajo Error)	Éxito; conserva la memoria de manera eficiente.

Recurrent Orthogonal Networks and Long-Memory Tasks

Stacked RNN



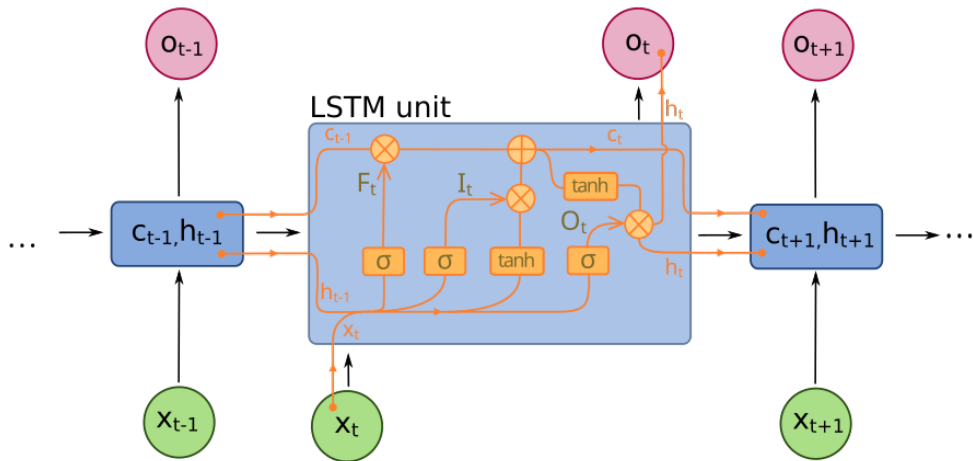
Bidirectional RNN



LSTM: Long Short-Term Memory

- Varias matrices de pesos funcionan como **compuertas lógicas** que controlan qué pasa con las entradas y la memoria:
 - **Update gate:**
 - Controla cómo y cuánto actualizar la memoria en este momento.
 - **Input gate:**
 - Determina qué tan importante es la entrada actual.
 - **Forget gate:**
 - Decide qué información olvidar.
 - **Output gate:**
 - Controla cuánta información es relevante para la salida.
- Utiliza dos estados: **cell state** y **hidden state** a lo largo del tiempo.

LSTM: Arquitectura de la celda



Ecuaciones de la LSTM

$$\mathbf{F}_t = \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{b}_f) \quad (\text{forget gate})$$

$$\mathbf{I}_t = \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{b}_i) \quad (\text{input gate})$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \mathbf{x}_t + \mathbf{U}_c \mathbf{h}_{t-1} + \mathbf{b}_c) \quad (\text{candidate cell state})$$

$$\mathbf{c}_t = \mathbf{F}_t \odot \mathbf{c}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{c}}_t \quad (\text{cell state update})$$

$$\mathbf{O}_t = \sigma(\mathbf{W}_o \mathbf{x}_t + \mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{b}_o) \quad (\text{output gate})$$

$$\mathbf{h}_t = \mathbf{O}_t \odot \tanh(\mathbf{c}_t) \quad (\text{hidden state output})$$

GRU: Gated Recurrent Unit

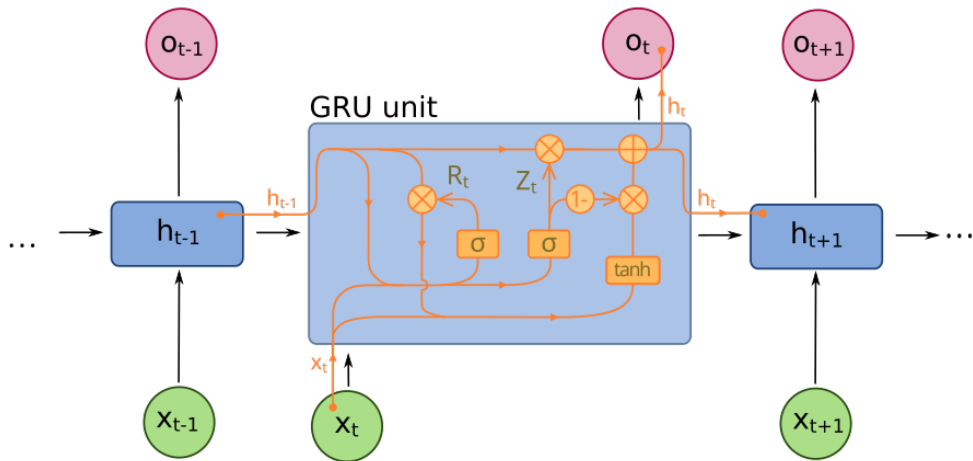
- Simplifica la celda de LSTM:

- Cambiando las **compuertas**.
- Esto simplifica la matemática.
- **Facilita el entrenamiento**.
- **Reduce el número de pesos**.

- **Utiliza:**

- **Input gate** → reemplazada por la **Reset gate**
Controla cuánta importancia tiene la entrada actual.
- **Update gate**: controla cuánto del estado previo se mantiene.

GRU: Arquitectura de la celda



Ecuaciones de la GRU

$$\mathbf{Z}_t = \sigma(\mathbf{W}_z \mathbf{x}_t + \mathbf{U}_z \mathbf{h}_{t-1} + \mathbf{b}_z) \quad (\text{update gate})$$

$$\mathbf{R}_t = \sigma(\mathbf{W}_r \mathbf{x}_t + \mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{b}_r) \quad (\text{reset gate})$$

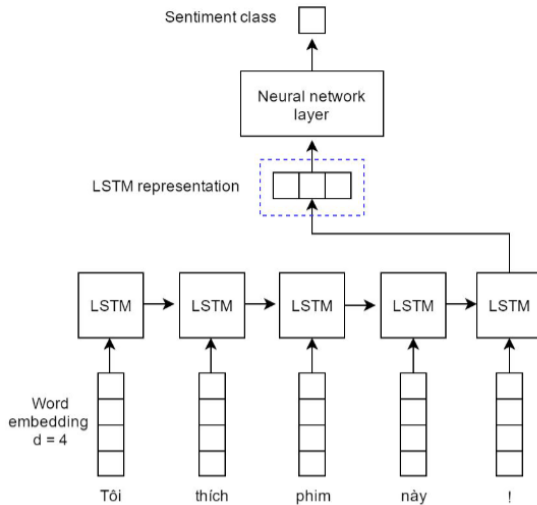
$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h (\mathbf{R}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h) \quad (\text{candidate hidden state})$$

$$\mathbf{h}_t = (\mathbf{1} - \mathbf{Z}_t) \odot \mathbf{h}_{t-1} + \mathbf{Z}_t \odot \tilde{\mathbf{h}}_t \quad (\text{hidden state update})$$

Intuición del funcionamiento del GRU

- **GRU** simplifica LSTM manteniendo solo dos compuertas:
 - **Reset gate ($\mathbf{R}_t$)**: decide cuánta información del pasado $\mathbf{h}_{t-1}$ debe ignorarse al generar el nuevo estado candidato.
 - **Update gate ($\mathbf{Z}_t$)**: controla el balance entre mantener la memoria previa y agregar nueva información.
- El flujo de información se interpreta así:
 - Si $\mathbf{R}_t$ es pequeño $\Rightarrow$ el modelo “resetea” la memoria.
 - Si $\mathbf{Z}_t$ es grande $\Rightarrow$ el modelo actualiza fuertemente el estado oculto con nueva información.
 - Si $\mathbf{Z}_t$ es pequeño $\Rightarrow$ el modelo mantiene la información previa $\mathbf{h}_{t-1}$.

Ejemplo de uso en clasificación



Resumen del ejemplo de clasificación

Elemento en la figura	Símbolo / Nivel	Descripción
Word embedding	$x_t \in \mathbb{R}^d$	Representación vectorial de la palabra en el paso t
LSTM cell	$(x_t, h_{t-1}, c_{t-1}) \rightarrow (h_t, c_t)$	Procesa entrada y contexto previo
LSTM representation	h_T (o agregación de $\{h_t\}$)	Resumen contextual de toda la secuencia
Neural network layer	$Wh_T + b$	Capa densa para clasificación
Sentiment class	$\arg \max(y)$	Predicción final (clase de sentimiento)

Bibliografía

- Scardapane, Simone. "Alice's Adventures in a Differentiable Wonderland–Volume I, A Tour of the Land." arXiv preprint arXiv:2404.17625 (2024).
- CS 230 - Deep Learning - Stanford
- Recurrent neural network - Wikipedia